

Reminder Famille

Application web de gestion des tâches familiales

Soutenance Titre Professionnel

Développeur Web et Web Mobile

Vladimir Rodzaevskiy — AFPA Marseille 2026

1. Le constat

Une famille moderne gère **beaucoup** de tâches récurrentes :

- 💰 Échéances financières (loyer, factures, impôts)
- 🏥 Rendez-vous médicaux
- 🚗 Maintenance véhicule (CT, vidange)
- 🧹 Corvées domestiques
- 📖 Suivi scolaire des enfants

Aujourd'hui, c'est dispersé entre post-it, WhatsApp, mémoire d'un seul parent → charge mentale.

2. Les solutions existantes échouent

Solution	Limitation
Todoist	Pensé entreprise, fonctions famille payantes (4€/mois/user)
Trello	Aucun modèle parent/enfant
Apple Reminders	iOS uniquement
Google Tasks	Pas de partage à plusieurs
Notion	Trop complexe, abonnement

Hypothèse : peut-on faire mieux avec des standards ouverts ?

3. Reminder Famille — la proposition

3 idées clés

1. Familles avec rôles **parent / enfant**
2. **Validation parent** quand enfant termine
3. **Notifications natives téléphone** via iCal — sans installer d'app

Livré

- Application full-stack fonctionnelle
- Multi-utilisateurs, multi-familles
- Mobile + desktop
- Mode sombre intégré
- 100% gratuit, auto-hébergeable

4. Stack technique

Front-end

- **React 18 + Vite 5**
- **React Router 6**
- **Context API** (Auth, Family, Theme)
- **Chart.js** pour stats
- CSS pur (glassmorphism + dark mode)

Back-end

- **Node.js 22 LTS**
- **Express 4**
- **mysql2** (SQL préparé, sans ORM)
- **bcrypt + JWT** (HS256)
- **ical-generator** (RFC 5545)

Base de données : MariaDB 10.11 · 5 tables · 6 migrations versionnées

5. Architecture en couches

```
Browser (React + Vite)
  | fetch JSON + JWT Bearer
  ▼
Express (Node.js 22)
  |— middleware : CORS, auth, rate-limit, family-access
  |— routes → controllers → validators → models
  |
  ▼
mysql2 (requêtes préparées)
  ▼
MariaDB 10.11 (ACID, FK strictes)
```

Principe : 1 entité = 1 fichier par couche → travail isolé + évolutif

6. Démonstration en direct ★

Passage à l'application

Scénario démo (12 étapes) :

1. Login Marie (parent admin) → Dashboard parent
2. Création tâche assignée à Léo
3. Calendrier mensuel + stats
4. Page Famille → invite_code + reset password
5. Page Profil → QR code iCal → scan iPhone
6. Logout, login Léo (enfant) → Dashboard enfant
7. Léo termine tâche → status `pending_review`
8. Re-login Marie → carte orange « À valider » → approve

7. Mission 1 — Auth & Familles

Périmètre : inscription, connexion, gestion familles, rôles, validation membres

Côté back

- JWT HS256 + bcrypt cost 10
- Middlewares composables :
 - `authRequired` →
 - `requireFamilyMember` →
 - `requireAdmin`
- **Last-admin protection** sur suppression
- Reset MDP par admin (sans email)

Côté front

- `AuthContext` global
- `ProtectedRoute` pour routes privées
- Persistance JWT en `localStorage`
- Réhydratation au reload via `/auth/me`

8. Mission 2 — Tâches & Validation parent

Workflow distinctif : enfant termine → parent valide

```
[Enfant] click "Terminer" sur tâche assignée
      ↓
    POST /api/tasks/:id/complete
      ↓
    status passe à 'pending_review'
      ↓
[Parent] voit carte orange sur Dashboard
      ↓
    POST /api/tasks/:id/approve ou /reject
      ↓
    once → completed | récurrente → due_date avancée (DATE_ADD)
```

21 scénarios de tests valident ce flux + permissions enfant/parent

9. Mission 3 — Export iCal ★

Le défi : notifier sur téléphone sans installer d'app

Approche	Problème
App native	2 stores, comptes développeur, mise à jour
Web Push	Limité iOS (PWA installée obligatoire)
Email / SMS	SMTP / coût / spam

✓ **Solution : standard iCalendar (RFC 5545)**

`webcal://api/calendar/<token>` → l'iPhone/Android **poll automatiquement** ce flux → **VALARM** = notification système 1j avant

Aucune app à installer. Fonctionne iOS + Android.

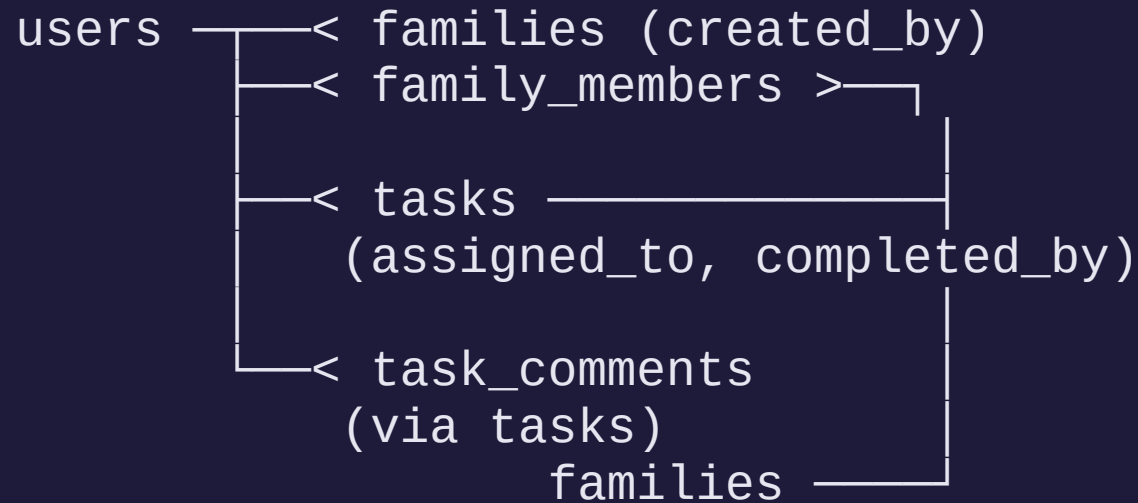
10. Sécurité — OWASP Top 10

Risque	Mesure
Injection SQL	100% requêtes préparées (? placeholders)
XSS	React échappement automatique (jamais dangerouslySetInnerHTML)
CSRF	JWT en header (pas de cookie)
Auth brute force	Rate limit 5/15min + message générique
Cryptographic failures	bcrypt cost 10 + JWT HS256
Broken access control	Middlewares composables + last-admin protection



Document détaillé : dossier-projet/securite.md

11. Conception BDD — 5 tables relationnelles



- InnoDB pour ACID + FK strictes
- **ON DELETE CASCADE/SET NULL/RESTRICT** selon sémantique
- **Index** sur user_id, family_id, due_date, status
- **ENUM MySQL** pour énumérations métier (catégorie, priorité, fréquence, statut, rôle)

12. Tests d'intégration — 57 / 57

```
$ bash tests/integration.sh
```

1. Authentification	11	✓
2. Familles	14	✓
3. Tâches CRUD & perms	11	✓
4. Validation parent	10	✓
5. Export iCal	7	✓
6. Statistiques	2	✓

```
Résultats : 57 passés / 57 total  
Tous les tests passent !
```

Script bash + `curl` + `jq` — réexécutable à chaque évolution

13. Le projet en chiffres

2 mois

développement solo

~6000

lignes de code

117

fichiers versionnés

57

tests d'intégration 

17

captures auto-générées

8

diagrammes UML/Merise

14. Choix discutés et évolutions

Compromis assumés

- **Pas d'ORM** → SQL visible, plus de boilerplate mais maîtrise
- **Pas de TypeScript** → temps gagné, risques runtime
- **Pas de tests unitaires** → 57 d'intégration à la place
- **Pas de déploiement** → priorité polish fonctionnel

Évolutions identifiées

- Tests unitaires Vitest
- CI/CD GitHub Actions
- Notifications email (Resend)
- Bot Telegram pour rappels temps réel
- PWA mode offline
- Déploiement Vercel + Fly.io

Merci pour votre attention

 **GitHub :**

github.com/Vladimir08880888/reminder-famille

github.com/Vladimir08880888/vladimir-rodzaevski-dossiers-pros-DWWM

 **Démo locale :**

`marie@famille.fr / motdepasse123`

 **Code MIT** — auto-hébergeable

Vos questions ?